server2 host log pipeline: install, select destinations, read logs, prepare AWS

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, and renders the AWS templates (LP-5) that the owner applies. Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

1. Install the pipeline; the default ships nothing

As root, run server2/deploy/logging/install.sh (with --collector-tarball for a local copy of the pinned archive, otherwise it downloads it). It checks the archive hash, installs the units and enables them. With no SERVER2_LOG_SINKS setting, start both units and check the collector.

```
$ bash server2/deploy/logging/install.sh --collector-tarball
    reports/lp-2977/vendor/vector-0.61.0-aarch64-unknown-linux-musl.tar.gz
install.sh: installed vector 0.61.0 (aarch64-unknown-linux-musl 17e017e 2026-10-29
    16:03:40.929269968)

$ systemctl start server2-logs.service server2.service; systemctl is-active server2.service
active

$ systemctl status server2-logs.service --no-pager | grep -E 'server2-logs.service
    -|Active:|Condition'
○ server2-logs.service - server2 log collector
     Active: inactive (dead) (Result: exec-condition) since Sat 2026-11-14 15:00:07 KST; 77ms ago
  Condition: start condition unmet at Sat 2026-11-14 15:00:07 KST; 135ms ago
    Process: 15185 ExecCondition=/usr/bin/python3 -I /opt/server2/logging/render.py --env-file
    /opt/server2/config/.env --state /var/lib/server2-logs --output /run/server2-logs/vector.json
    --version-file /opt/server2/logging/collector.json (code=exited, status=1/FAILURE)
Nov 14 15:00:07 lima-lp2977 systemd[1]: Starting server2-logs.service - server2 log collector...
Nov 14 15:00:07 lima-lp2977 systemd[1]: Condition check resulted in server2-logs.service - server2
    log collector being skipped.

$ journalctl --namespace=server2 -u server2-logs.service -o cat
server2 log collector: only the local journal is selected; nothing is shipped
```

install.sh reports the collector version. server2.service is active; server2-logs.service is skipped by its start condition and says why in the server2 journal.

Expected result: The server runs and writes to the local journal namespace server2. The collector is skipped (start condition unmet), so nothing leaves the host.

server2 host log pipeline: install, select destinations, read logs, prepare AWS

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, and renders the AWS templates (LP-5) that the owner applies. Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

2. A mistyped setting fails only the collector

Set SERVER2_LOG_SINKS in /opt/server2/config/.env (here with a typo, clodwatch) and start the units. Read the collector's message in the server2 journal namespace, fix the value, and start the collector again.

```
$ grep '^SERVER2_LOG' /opt/server2/config/.env
SERVER2_LOG_SINKS=s3,clodwatch
SERVER2_LOG_S3_PREFIX=logs/server2

$ systemctl start server2-logs.service server2.service; systemctl is-active server2-logs.service
    server2.service
Job for server2-logs.service failed because the control process exited with error code.
See "systemctl status server2-logs.service" and "journalctl -xeu server2-logs.service" for details.
failed
active

$ journalctl --namespace=server2 -u server2-logs.service -o cat
server2 log collector: invalid setting SERVER2_LOG_SINKS: unknown destination; allowed: local, s3,
    cloudwatch; the collector is not started

$ curl -s -o /dev/null -w 'health %{http_code}\n' http://127.0.0.1:18090/health
health 200
```

The collector is failed and names SERVER2_LOG_SINKS and the allowed values, without repeating the mistyped value. The server is active and /health answers 200.

Expected result: server2-logs.service is failed without restarting in a loop; server2.service keeps serving.

server2 host log pipeline: install, select destinations, read logs, prepare AWS

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, and renders the AWS templates (LP-5) that the owner applies. Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

3. Ship to S3 and CloudWatch Logs with different levels

Set the destinations, the per-destination levels, the S3 prefix and an existing CloudWatch log group in .env, then start (or restart) the collector. On EC2 the credentials come from the instance role.

```
$ grep '^SERVER2_LOG' /opt/server2/config/.env
SERVER2_LOG_SINKS=local,s3,cloudwatch
SERVER2_LOG_S3_LEVEL=info
SERVER2_LOG_CLOUDWATCH_LEVEL=warn
SERVER2_LOG_S3_PREFIX=logs/server2
SERVER2_LOG_CLOUDWATCH_GROUP=/server2/production

$ systemctl start server2-logs.service server2.service; sleep 2; systemctl is-active
    server2-logs.service server2.service
active
active

$ journalctl --namespace=server2 -u server2-logs.service -o cat | grep collector.started | jq -c .
{"boot_id":"f285e857880a4a03b545bbefa2e2a1ef","collector_schema":1,"collector_version":"vector 0.61.
    0","event":"collector.started","instance_id":"b09bf0476120469f9b0cd4349a3edcd1","level":"info","
    sinks":["local","s3","cloudwatch"],"timestamp":"2026-11-14T06:00:13.140800Z"}
```

Both units are active. The collector's first record names its version, the active destinations, the host (machine ID until LP-3) and the boot.

Expected result: The collector start record appears in the journal and is shipped to every remote destination.

server2 host log pipeline: install, select destinations, read logs, prepare AWS

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, and renders the AWS templates (LP-5) that the owner applies. Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

4. Where records arrive

Serve requests, then stop the server and, a few seconds later, the collector (until LP-4 adds a drain step, stopping both at once can leave the server's last records in the local journal only). Look at what arrived: S3 keys under the prefix and CloudWatch streams in the group.

```
# What the stand-in received in place of S3 (keys) and CloudWatch Logs (streams)
$ cd /var/tmp/manual-standin/s3 && find . -type f | sed 's#^./##'
logs/server2/collector/dt=2026-11-
    14/hour=06/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef-20261114T060021Z-
    d559a150-0030-4596-b9dd-bd087ef75553.ndjson.gz
logs/server2/server/dt=2026-11-
    14/hour=06/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef-20261114T060021Z-
    1bfd615c-2b1c-4eb1-8234-313ae4e0b05b.ndjson.gz

$ cd /var/tmp/manual-standin/cloudwatch/server2/production && find . -type f | sed 's#^./##;
    s#.jsonl$##'
collector/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef
server/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef
```

S3 keys and CloudWatch streams carry the source (server or collector), the host ID and the boot ID; S3 keys also carry the UTC date and hour and a unique suffix.

Expected result: Server and collector records are kept apart, and records from different hosts or boots never share a stream or key.

server2 host log pipeline: install, select destinations, read logs, prepare AWS

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, and renders the AWS templates (LP-5) that the owner applies. Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

5. Read each destination

Read the local journal with journalctl --namespace=server2, an S3 object with gzip -dc (after aws s3 cp), and the CloudWatch stream (aws logs tail on EC2). Filter with jq.

```
$ journalctl --namespace=server2 -u server2.service -o cat | grep -v '^{'
server2 public=127.0.0.1:18090 metrics=127.0.0.1:18091

$ journalctl --namespace=server2 -u server2.service -o cat | grep '^{' | jq -c
    'select(.level=="warn") | {timestamp,level,component,stage,status} | with_entries(select(.value
    != null))'
{"timestamp":"2026-11-
    14T06:00:16.245834599Z","level":"warn","component":"gateway","stage":"provider_http"}
{"timestamp":"2026-11-
    14T06:00:16.248366929Z","level":"warn","component":"http","stage":"request","status":501}

# S3 object (gzip NDJSON), read as with: aws s3 cp s3://BUCKET/KEY - | gzip -dc
$ gzip -dc /var/tmp/manual-standin/s3/logs/server2/server/dt=2026-11-
    14/hour=06/b09bf0476120469f9b0cd4349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef-20261114T060021Z-
    1bfd615c-2b1c-4eb1-8234-313ae4e0b05b.ndjson.gz | jq -c '{level,event,component,stage,status} |
    with_entries(select(.value != null))' | head -n 8
{"level":"info","event":"operation.action","component":"process","stage":"startup"}
{"level":"info","event":"operation.action","component":"process","stage":"bind"}
{"level":"info","event":"operation.started","component":"http","stage":"request"}
{"level":"info","event":"operation.started","component":"gateway","stage":"request"}
{"level":"warn","event":"operation.finished","component":"gateway","stage":"provider_http"}
{"level":"info","event":"operation.finished","component":"gateway","stage":"request"}
{"level":"warn","event":"operation.finished","component":"http","stage":"request","status":501}
{"level":"info","event":"operation.action","component":"process","stage":"shutdown"}

# CloudWatch Logs stream server/<instance>/<boot>, read as with: aws logs tail GROUP --log-stream-
    name-prefix server/
$ jq -r .message /var/tmp/manual-standin/cloudwatch/server2/production/server/b09bf0476120469f9b0cd4
    349a3edcd1/f285e857880a4a03b545bbefa2e2a1ef.jsonl | jq -c '{level,event,component,stage,status}
    | with_entries(select(.value != null))'
{"level":"warn","event":"operation.finished","component":"gateway","stage":"provider_http"}
{"level":"warn","event":"operation.finished","component":"http","stage":"request","status":501}
```

The readiness line stays in the journal only. S3 (level info) holds the info and warn records; CloudWatch (level warn) holds only the two warn records of the HTTP 501 request.

Expected result: Each destination holds exactly the server records at or above its level, as the original lines.

server2 host log pipeline: install, select destinations, read logs, prepare AWS

An operator installs the server2 log collector on a Linux host, keeps the default local journal, recovers from a mistyped setting, ships the log to S3 and CloudWatch Logs with different levels, reads each destination, and renders the AWS templates (LP-5) that the owner applies. Captured on 2026-11-14 in a disposable Ubuntu 24.04 VM (systemd 255) with the pinned Vector 0.61.0 collector and a server2 release binary; S3 and CloudWatch Logs were the loopback stand-in server2/deploy/logging/standin.py, reached through the real regional AWS host names. All values are synthetic. Full reference: docs/operations/server2-logging.md, section Host log pipeline.

6. Prepare the AWS templates (LP-5)

Render the role policy, bucket lifecycle and log group templates for the actual bucket, prefix, group, region and account with server2/deploy/logging/aws_templates.py. Then apply them with administrator credentials using the commands in the guide (section Prepare AWS for the destinations); put-bucket-lifecycle-configuration replaces the whole configuration, so read it first. The capture shows synthetic names; nothing was applied to AWS.

```
$ python3 -I server2/deploy/logging/aws_templates.py --bucket tw-server2-123456789012-apne2 --prefix
     raw/v2/logs --group /server2/production --region ap-northeast-2 --account 123456789012 --out
    $OUT
aws_templates: SERVER2_LOG_S3_PREFIX: must not start inside a weather data prefix: raw, index,
    packs, summary, state, budgets, geocode

$ python3 -I server2/deploy/logging/aws_templates.py --bucket tw-server2-123456789012-apne2 --prefix
     logs/server2 --group /server2/production --region ap-northeast-2 --account 123456789012 --out
    $OUT
aws_templates: rendered 3 files into $OUT

$ jq -c '.Statement[] | {Sid, Action, Resource}' $OUT/role-policy.json
{"Sid":"PutLogObjects","Action":["s3:PutObject"],"Resource":"arn:aws:s3:::tw-
    server2-123456789012-apne2/logs/server2/*"}
{"Sid":"WriteLogStreams","Action":["logs:CreateLogStream","logs:PutLogEvents"],"Resource":"arn:aws:l
    ogs:ap-northeast-2:123456789012:log-group:/server2/production:log-stream:*"}
{"Sid":"FindLogStreams","Action":["logs:DescribeLogStreams"],"Resource":["arn:aws:logs:ap-
    northeast-2:123456789012:log-group:/server2/production","arn:aws:logs:ap-
    northeast-2:123456789012:log-group:/server2/production:*"]}

$ jq -c '.Rules[] | {ID, Filter, Expiration, NoncurrentVersionExpiration,
    AbortIncompleteMultipartUpload} | with_entries(select(.value != null))' $OUT/bucket-
    lifecycle.json
{"ID":"server2-logs-expire","Filter":{"Prefix":"logs/server2/"},"Expiration":{"Days":90},"Noncurrent
    VersionExpiration":{"NoncurrentDays":1},"AbortIncompleteMultipartUpload":{"DaysAfterInitiation":
    1}}
{"ID":"server2-logs-delete-
    markers","Filter":{"Prefix":"logs/server2/"},"Expiration":{"ExpiredObjectDeleteMarker":true}}

$ jq -c . $OUT/log-group.json
{"logGroupName":"/server2/production","logGroupClass":"STANDARD","retentionInDays":30}
```

A prefix inside a weather data prefix is refused and nothing is written. The rendered policy grants only PutObject under the log prefix, stream creation, event writing and stream lookup in the one group; the lifecycle covers only the log prefix; the group keeps 30 days.

Expected result: Three JSON files without placeholders, ready for the owner to apply and read back.